

Setor de Educação Profissional e Tecnológica Tecnologia em Análise e Desenvolvimento de Sistemas

DS122 - Desenvolvimento de Aplicações Web 1

03 - Protocolo HTTP

Prof. Alex Kutzke

Agosto de 2026

Objetivos da aula

Ao final desta aula você deve ser capaz de:

1. Descrever a estrutura de uma mensagem HTTP byte a byte;
2. Explicar o que significa o protocolo ser *sem estado* e as consequências disso;
3. Identificar os três lugares por onde o cliente envia parâmetros: caminho, consulta e corpo;
4. Escolher o método HTTP correto, justificando com segurança e idempotência;
5. Interpretar códigos de status e cabeçalhos;
6. Construir requisições manualmente com curl;
7. Atuar como servidor, respondendo à mão a uma requisição do navegador.

Roteiro

Fundamentos

A mensagem

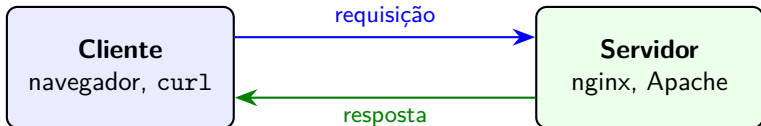
Semântica

Além de uma requisição

Prática

Exercícios

Modelo cliente-servidor



- **Cliente:** inicia a comunicação. Envia uma requisição e aguarda.
- **Servidor:** nunca inicia. Fica escutando e responde.

No modelo **desktop** existe um ator só: o programa roda na máquina do usuário, lê o teclado, escreve na tela, acessa arquivos locais.

O servidor nunca inicia a comunicação

O servidor não tem como avisar espontaneamente o navegador de que algo mudou. Toda informação que chega ao cliente chega **porque o cliente pediu**.

Uma página que precisa exibir dados novos sem interação do usuário só tem uma saída dentro do HTTP: perguntar de novo, de tempos em tempos. Contornar essa limitação exige tecnologias construídas por cima do protocolo, como WebSocket e *Server-Sent Events*, fora do escopo desta disciplina.

O que é um protocolo

Duas máquinas conectadas conseguem trocar bytes. Porém, isso geralmente não é suficiente: elas precisam concordar sobre o **significado** desses bytes.

- Quem manda primeiro?
- Como se pede um documento?
- Como se responde que o documento não existe?

Um **protocolo** é esse acordo.

Na web, o protocolo é o **HTTP** (*HyperText Transfer Protocol*), criado por Tim Berners-Lee no início dos anos 1990.

Onde o HTTP fica

Aplicação: HTTP, DNS, SMTP

Transporte: TCP, UDP

Rede: IP

Enlace: Ethernet, Wi-Fi

O HTTP **não** se preocupa em fazer os bytes chegarem em ordem e sem perdas. Isso é tarefa do TCP, abaixo dele. O HTTP assume um canal confiável e define o que trafega por ele.

RFCs

As especificações da Internet são publicadas como **RFCs** (*Request for Comments*), numeradas e mantidas pela IETF.

Uma RFC nunca é editada depois de publicada

Quando a especificação muda, publica-se uma RFC nova que **torna obsoleta** (*obsoletes*) a anterior.

O HTTP/1.1 foi definido pela RFC 2616, de 1999. Ela está **obsoleta desde 2014**, e a maior parte do material antigo na Internet ainda a cita.

Especificações vigentes

RFC	Ano	Define
9110	2022	Semântica do HTTP (métodos, status, cabeçalhos), comum a todas as versões
9111	2022	Cache
9112	2022	Sintaxe do HTTP/1.1
9113	2022	HTTP/2
9114	2022	HTTP/3

Ao abrir uma RFC, procure no topo os campos `Obsoletes` e `Obsoleted by`. É assim que se sabe se o documento ainda vale.

Duas formas de ver uma mensagem real

Nos próximos minutos você vai olhar mensagens HTTP **reais**, de dois jeitos diferentes:

1. pelo terminal, com o `curl`;
2. pelo navegador, com as ferramentas de desenvolvimento.

Os detalhes não precisam ser entendidos ainda. Cada um deles é uma seção desta aula, e as duas ferramentas voltam em detalhe na parte prática.

No terminal

```
curl -v https://example.com
```

- > são as linhas que o curl **enviou**;
- < são as linhas que o servidor **devolveu**;
- * são comentários do próprio curl.

Quatro coisas a notar: a conversa é **texto**; tem uma primeira linha diferente das outras; depois vem uma pilha de pares Nome: valor; por fim uma linha em branco, e só então o HTML.

No navegador

1. Abra qualquer site;
2. F12 ou Ctrl+Shift+I;
3. aba **Rede** (*Network*);
4. recarregue a página com Ctrl+R;
5. clique em qualquer linha da lista que aparecer.

Na aba **Cabeçalhos** (*Headers*) você encontra os mesmos dois blocos: o que o navegador mandou e o que o servidor respondeu.

É a mesma conversa que o curl mostrou. Muda a apresentação.

A mesma mensagem nas duas ferramentas

	No curl -v	No navegador
Endereço pedido	1ª linha após >	<i>Request URL</i>
Verbo	GET, no início	<i>Request Method</i>
Deu certo?	1ª linha após <	<i>Status Code</i>
Metadados enviados	pares após >	<i>Request Headers</i>
Metadados recebidos	pares após <	<i>Response Headers</i>
Conteúdo	depois da linha em branco	<i>aba Resposta</i>

As seis linhas da tabela são, nesta ordem, o roteiro das próximas seções.

Anatomia de uma mensagem

Toda mensagem HTTP/1.1, de requisição ou de resposta, tem a mesma forma:

```
<linha inicial>
<Nome-do-cabecalho>: <valor>
<Nome-do-cabecalho>: <valor>
<linha em branco>
<corpo, opcional>
```

E o mais importante: **é texto**. Você pode digitá-la à mão.

CRLF e a linha em branco

CRLF

Cada linha termina com **dois** caracteres: `\r\n` (retorno de carro e nova linha). Não é apenas `\n`, como é usual em arquivos de texto no Linux. Terminador errado, mensagem rejeitada.

A linha em branco

Separa os cabeçalhos do corpo. É **obrigatória**, mesmo quando não há corpo.

É por isso que, ao digitar uma requisição à mão, você pressiona Enter duas vezes.

Requisição

```
GET /docs/index.html HTTP/1.1
```

```
Host: www.example.com
```

```
User-Agent: curl/8.5.0
```

```
Accept: text/html
```

A **linha inicial** tem três partes separadas por espaço:

1. o **método** (GET): o que se quer fazer;
2. o **alvo** (/docs/index.html): o caminho do recurso;
3. a **versão** do protocolo (HTTP/1.1).

Repare: o alvo **não contém o nome do site**.

Resposta

HTTP/1.1 200 OK

Date: Mon, 23 May 2026 22:38:34 GMT

Server: nginx

Content-Type: text/html; charset=UTF-8

Content-Length: 47

<html>

<body>

<p>Ola mundo</p>

</body>

</html>

1. a **versão** do protocolo;
2. o **código de status** (200), para o programa;
3. a **frase de motivo** (OK), para o humano, ignorada pelo programa.

O cabeçalho Host

Host é obrigatório em HTTP/1.1 e não existia em HTTP/1.0. A razão é econômica.

Um único servidor, com um único endereço IP, pode hospedar centenas de sites. Quando a requisição chega, o servidor precisa decidir **qual site entregar**, e a única informação disponível é o cabeçalho Host.

Chama-se **hospedagem virtual por nome** (*name-based virtual hosting*).

DNS, TCP e TLS antes do HTTP

1. O navegador consulta o **DNS** para descobrir o IP de `www.example.com`;
2. Abre uma **conexão TCP** com esse IP;
3. (se HTTPS) executa o **handshake TLS**;
4. Envia a requisição, informando **dentro dela** com qual nome pretendia falar.

O passo 4 é o que torna a hospedagem virtual possível.

Onde a mensagem termina

Pergunta que parece trivial e não é:

Recebida a última linha de cabeçalho, como o cliente sabe **em que ponto** a resposta acabou?

O TCP entrega um fluxo contínuo de bytes. Ele **não** sinaliza fronteiras de mensagem.

O HTTP resolve isso de três formas.

1. Content-Length

O servidor informa o tamanho exato do corpo em bytes. O cliente conta.

```
HTTP/1.1 200 OK
```

```
Content-Length: 47
```

Exige que o servidor conheça o tamanho total **antes** de começar a enviar.

2. Transfer-Encoding: chunked

O corpo vai em blocos, cada um precedido pelo seu tamanho em hexadecimal. Um bloco de tamanho 0 encerra a mensagem.

```
HTTP/1.1 200 OK
Transfer-Encoding: chunked
```

```
7
01a mun
3
do!
0
```

Corpo reconstruído: 01a mundo.

Permite responder **antes** de saber o tamanho final. É a base do envio contínuo de dados (*streaming*).

3. Fechamento da conexão

O servidor envia `Connection: close`, transmite o corpo e encerra a conexão TCP. O fim da conexão é o fim da mensagem.

Era o mecanismo do HTTP/1.0. É ineficiente, porque impede reaproveitar a conexão.

Requisição pendurada e resposta truncada

Requisição que fica pendurada indefinidamente: o cliente ainda espera bytes prometidos por um `Content-Length` maior que o corpo real.

Resposta truncada: o contrário.

Anatomia de uma URL

`https://usuario@www.example.com:8080/produtos/lista.php?cat=livros&pag=2#topo`

esquema userinfo host porta caminho consulta fragmento

Sete partes. Duas delas, **consulta** e **fragmento**, são as que mais confundem, e são justamente as que interessam nesta aula.

As partes da URL

esquema	O protocolo: http, https, mailto, file. Determina tudo o que vem depois.
userinfo	Credencial embutida. Em desuso no HTTP e desencorajado por segurança.
host	Nome de domínio ou IP do servidor.
porta	Porta TCP. Omitida, vale 80 para http e 443 para https.
caminho	Identifica o recurso dentro do servidor.
consulta	Pares chave=valor separados por &, iniciados por ?. É o que o PHP lerá em \$_GET.
fragmento	Iniciado por #. Identifica parte interna do documento.

O fragmento nunca é enviado ao servidor

`https://site.com/pagina#secao`

O trecho `#secao` é processado **inteiramente pelo navegador**, que o usa para rolar a página até o elemento correspondente.

O servidor **jamais** toma conhecimento dele.

Você vai comprovar isso experimentalmente na parte prática desta aula.

Codificação percentual

Espaços, acentos e os próprios caracteres ?, & e # não podem aparecer cruamente numa URL.

São substituídos por % seguido do valor hexadecimal de cada byte na codificação UTF-8.

Caractere	Codificado
espaço	%20
á	%C3%A1
&	%26
#	%23

A busca por análise e projeto vira:

q=an%C3%A1lise%20e%20projeto

Note que á ocupa **dois** bytes em UTF-8.

Passagem de parâmetros

Até agora o cliente só pediu: “me entregue o recurso deste caminho”.

Mas uma aplicação web precisa que o cliente **envie dados**:

- o termo digitado na busca;
- a página da listagem que se quer ver;
- o login e a senha;
- os campos de um cadastro.

Sem isso não existe aplicação, só um servidor de arquivos. Toda a disciplina, daqui em diante, gira em torno disso.

Caminho, consulta e corpo

A mensagem HTTP tem três lugares onde dados cabem.

Lugar	Exemplo	Onde fica na mensagem
Caminho	/produtos/42	linha inicial
Consulta	/busca?q=teclado	linha inicial, após ?
Corpo	q=teclado	depois da linha em branco

Cabeçalhos também carregam dados (Cookie, Authorization), mas são metadados, e não parâmetros da operação.

Parâmetros na própria URL

/busca?q=teclado&pag=2&ordem=preco

Regras da *query string*:

- começa no primeiro ?;
- cada parâmetro é um par chave=valor;
- pares são separados por &;
- caracteres especiais vão codificados: q=ar%20condicionado.

O servidor recebe isto e monta uma estrutura de chave e valor. Em PHP:

```
$_GET['q']      // "teclado"
$_GET['pag']    // "2"
```

Parâmetros no corpo da mensagem

```
POST /busca HTTP/1.1
```

```
Host: loja.com
```

```
Content-Type: application/x-www-form-urlencoded
```

```
Content-Length: 15
```

```
q=teclado&pag=2
```

Exatamente a mesma gramática da consulta, só que depois da linha em branco. O nome do formato diz isso: `x-www-form-urlencoded`, ou seja, “codificado como numa URL”.

Em PHP, muda apenas o nome da variável: `$_POST['q']`.

O formulário HTML

Você raramente monta a URL à mão. Quem monta é o formulário HTML.

```
<form action="/busca" method="get">
  <input name="q">
  <input name="pag">
</form>
```

Com method="get", o navegador coloca os campos na **consulta**:
/busca?q=teclado&pag=2

Com method="post", coloca os mesmos campos no **corpo**.

GET ou POST?

Se os dois transportam os mesmos dados, qual escolher, e por quê?

Métodos

O método é o verbo da requisição.

GET	Solicita a representação de um recurso.
POST	Envia dados para serem processados pelo recurso.
PUT	Substitui integralmente o recurso.
DELETE	Remove o recurso.
HEAD	Igual ao GET, mas devolve só os cabeçalhos.
OPTIONS	Pergunta quais métodos o servidor aceita.
PATCH	Altera parcialmente o recurso.

GET e POST não se distinguem pelo lugar dos dados

“GET manda os dados pela URL e POST manda pelo corpo.”

A formulação é verdadeira, mas descreve o **efeito**, não a **causa**, e pode levar a decisões erradas de projeto.

A distinção que importa está em duas propriedades definidas na RFC 9110.

Seguro e idempotente

Método seguro (*safe*)

Não altera o estado do servidor. É apenas leitura.

Método idempotente

Executá-lo uma vez ou N vezes seguidas produz o **mesmo estado final** no servidor.

Método	Seguro	Idempotente
GET	sim	sim
HEAD	sim	sim
PUT	não	sim
DELETE	não	sim
POST	não	não

DELETE não é seguro, mas é idempotente: apagar o mesmo recurso duas vezes deixa o servidor no mesmo estado, ainda que a segunda resposta seja 404. POST não é, porque enviar duas vezes o formulário de compra cria duas compras.

Consequências no navegador

- Ele pode buscar antecipadamente um link, porque GET é seguro e não causa dano;
- Ele recarrega um GET sem perguntar nada, e exige confirmação antes de reenviar um POST;
- Caches e servidores intermediários armazenam respostas de GET, e nunca de POST;
- O histórico guarda a URL, portanto guarda os dados de um GET.

Senhas e links que apagam

Senha não viaja por GET

Mesmo em HTTPS. Ela ficaria no histórico do navegador e nos registros de acesso do servidor.

Link que apaga registro não é GET

Qualquer mecanismo que percorra links da página o executaria sem intenção.

Códigos de status: as classes

O primeiro dígito define a classe.

1xx	Informativo. Processamento em andamento.
2xx	Sucesso.
3xx	Redirecionamento. O cliente precisa de uma ação adicional.
4xx	Erro do cliente . A requisição está errada.
5xx	Erro do servidor . A requisição estava certa.

A fronteira entre 4xx e 5xx indica onde procurar o erro

Ao depurar um sistema, 4xx manda olhar o código que fez a requisição, e 5xx manda olhar o servidor.

Códigos frequentes

Código	Nome	Quando ocorre
200	OK	Sucesso, com corpo.
201	Created	Recurso criado. Acompanha Location.
204	No Content	Sucesso sem corpo. Não pode ter corpo.
301	Moved Permanently	Mudou definitivamente. O navegador memoriza.
302	Found	Mudou temporariamente. Não memoriza.
304	Not Modified	Não mudou desde a última vez. Sem corpo.
400	Bad Request	Requisição malformada.
401	Unauthorized	Falta autenticação. Acompanha WWW-Authenticate.
403	Forbidden	Entendeu, sabe quem você é, e recusa.
404	Not Found	O recurso não existe naquele caminho.
405	Method Not Allowed	Recurso existe, método não é aceito. Acompanha Allow.
500	Internal Server Error	Falha no servidor. Em PHP, erro fatal no script.

401 contra 403, 301 contra 302

401 contra 403

401: “identifique-se”.

403: “já sei quem você é e ainda assim não pode”.

301 contra 302

O 301 é memorizado pelo navegador de forma persistente.

Publicar um 301 errado é **difícil de reverter**: os navegadores dos usuários continuarão indo ao endereço antigo mesmo depois da correção.

Cabeçalhos

Pares Nome: valor que carregam os metadados da mensagem. O nome não diferencia maiúsculas de minúsculas. Existem centenas.

Enviados pelo cliente

Host Nome do site.
Obrigatório.

User-Agent Identifica o programa.

Accept Tipos que sabe processar.

Accept-Language Idiomas preferidos.

Accept-Encoding Compressões.

Content-Type Formato do corpo.

Content-Length Tamanho do corpo.

Cookie Dados devolvidos.

Authorization Credenciais.

Enviados pelo servidor

Content-Type Formato do corpo.

Content-Length Tamanho do corpo.

Location Destino em 3xx.

Set-Cookie Pede para armazenar.

Cache-Control Regras de cache.

ETag Versão atual do recurso.

Server Software servidor.

Negociação de conteúdo

Os cabeçalhos Accept* permitem que cliente e servidor combinem o formato da resposta.

O cliente **declara** o que aceita. O servidor **escolhe** entre o que sabe produzir.

A mesma URL pode devolver HTML para um navegador e JSON para um programa.

Nem todo servidor negocia

Negociação de conteúdo é um recurso do protocolo. Cabe a cada servidor implementá-la ou não. Muitos escolhem o formato pelo caminho da URL.

Content-Type

Informa como interpretar os bytes do corpo. Segue o padrão MIME.

text/html	Documento HTML
text/css	Folha de estilo
application/javascript	Script
application/json	Dados em JSON
image/png	Imagem PNG
application/x-www-form-urlencoded	Formulário HTML padrão
multipart/form-data	Formulário com upload de arquivo

charset e os acentos quebrados

Content-Type: text/html; charset=UTF-8

O parâmetro charset diz que os bytes devem ser lidos como UTF-8.

A causa mais comum de ÃãÃço na tela

Um charset errado ou ausente.

O problema quase nunca está no arquivo. Está na **declaração**.

Protocolo sem estado

Stateless

Cada requisição é interpretada de forma isolada. O servidor não guarda nenhuma lembrança das requisições anteriores.

Duas requisições do mesmo usuário, com um segundo de intervalo, são para o servidor **dois eventos sem relação entre si**.

Escala, e o problema do usuário logado

- Sem estado, **qualquer** servidor em cluster (ou “fazenda de servidores”) pode atender **qualquer** requisição;
- Derrubar um servidor não derruba as sessões em curso.

Foi o que permitiu a web escalar. Mas:

Se o servidor não se lembra de nada,
como existe “usuário logado”?

A única saída é o **cliente reenviar**, em toda requisição, alguma informação que permita ao servidor reconstruir o contexto.

Cookies, sessões e tokens

- Cookies** O servidor responde com Set-Cookie:
nome=valor. O navegador armazena e passa a
incluir Cookie: nome=valor em toda requisição
para aquele domínio. É apenas texto em cabeçalho.
- Sessões** O cookie guarda só um identificador aleatório. Os
dados ficam no servidor, associados a esse
identificador. É o modelo que usaremos com PHP.
- Tokens** Uma credencial assinada, enviada em
Authorization. Comum em APIs.

Todos resolvem o mesmo problema pela mesma via: **fazer o cliente carregar o contexto**, porque o protocolo não o carrega.

HTTP/1.0 e HTTP/1.1

HTTP/1.0

Uma conexão TCP **por recurso**. Uma página com trinta imagens exigia trinta conexões, cada uma com o custo de abertura e fechamento.

HTTP/1.1

Conexões persistentes: a conexão TCP permanece aberta e várias requisições são feitas em sequência por ela.

Bloqueio de cabeça de fila

No HTTP/1.1 as requisições são atendidas **em ordem**. Uma resposta lenta bloqueia todas as seguintes na mesma conexão.

Chama-se **bloqueio de cabeça de fila** (*head-of-line blocking*).

Os navegadores contornavam abrindo cerca de **seis conexões paralelas** por domínio.

HTTP/2 (2015) e HTTP/3 (2022)

HTTP/2

Mensagens deixam de ser texto e passam a ser **binárias**, em quadros. Traz **multiplexação**: várias requisições e respostas trafegam simultaneamente e intercaladas na mesma conexão, eliminando o bloqueio no nível do HTTP. Comprime cabeçalhos com HPACK.

HTTP/3

Substitui o TCP pelo **QUIC**, que roda sobre UDP e implementa confiabilidade por conta própria. Elimina o bloqueio de cabeça de fila que **persistia na camada TCP** e funde o estabelecimento de conexão com o handshake criptográfico, reduzindo as idas e vindas até o primeiro byte.

A semântica é a mesma nas três versões

Métodos, códigos de status e cabeçalhos são definidos na RFC 9110 e valem para HTTP/1.1, HTTP/2 e HTTP/3.

O que muda é a **codificação** e o **transporte**.

HTTP/1.1 é o único legível por humanos

Porque ele é texto e pode ser digitado à mão.

A versão 2 é a mesma conversa, em formato ilegível para humanos.

HTTPS

HTTPS é HTTP transportado dentro de um túnel **TLS** (*Transport Layer Security*, sucessor do SSL).

O protocolo HTTP **não muda**. As mesmas mensagens de texto trafegam, agora cifradas.

No *handshake*, cliente e servidor negociam algoritmos, o servidor apresenta seu **certificado digital**, e ambos derivam uma chave de sessão usada para cifrar o restante da conversa.

O que o TLS garante

Confidencialidade Quem observa a rede não lê o conteúdo das mensagens: caminho da URL, cabeçalhos, cookies e corpo.

Integridade Alterações no tráfego são detectadas.

Autenticação do servidor O certificado, assinado por uma autoridade certificadora reconhecida pelo navegador, comprova que o servidor controla aquele nome de domínio.

O que o TLS não garante

- **Não esconde com quem você fala.** O IP de destino é visível, e o nome do host viaja em claro no handshake (campo SNI);
- **Não atesta idoneidade.** Certificados são gratuitos e obtidos em minutos. Um site fraudulento tem cadeado igual ao de um banco;
- **Não protege o dado depois que ele chega.** Segurança do banco de dados, das senhas e do código do servidor são problemas separados.

Conteúdo misto

Página HTTPS que carrega script por HTTP: o atacante troca o script e controla a página inteira. A garantia é anulada pelo elo mais fraco, e por isso os navegadores bloqueiam a carga.

Origem

esquema + host + porta

Duas URLs são da mesma origem somente se as **três** coincidirem.

URL A	URL B	Mesma origem?
https://site.com/a	https://site.com/b/c	sim, caminho não conta
http://site.com	https://site.com	não, esquema difere
https://site.com	https://api.site.com	não, host difere
http://site.com	http://site.com:8080	não, porta difere

Política de mesma origem e CORS

Regra imposta pelo **navegador**: JavaScript executando numa origem não pode ler a resposta de uma requisição feita a outra origem. Sem ela, uma página maliciosa faria requisições ao seu banco em outra aba, aproveitando os cookies já armazenados, e leria o resultado.

CORS (*Cross-Origin Resource Sharing*) é como o servidor de destino autoriza essa leitura, pelo cabeçalho `Access-Control-Allow-Origin`. Para requisições que possam alterar dados, o navegador antes envia uma verificação prévia com `OPTIONS`.

Quem impõe e quem autoriza

1. A restrição é do **navegador**, não do protocolo. O `curl` acessa qualquer origem. Por isso uma URL funciona no terminal e falha no console.
2. Quem autoriza é o **servidor de destino**. Não há nada no seu JavaScript que contorne a recusa.

Cache por prazo

Cache-Control: max-age=3600

O servidor declara que a resposta vale por 3600 segundos.

Durante esse período o navegador usa a cópia local e **não faz requisição nenhuma**.

Requisição condicional

Vencido o prazo, o cliente não precisa baixar tudo de novo. Ele **pergunta se mudou**, usando um identificador de versão que guardou:

ETag Etiqueta opaca que identifica a versão do recurso.

Servidor: ETag: "abc123"

Cliente reenvia: If-None-Match: "abc123"

Last-Modified Data da última alteração.

Cliente reenvia: If-Modified-Since: <data>

Se não mudou, o servidor responde 304 Not Modified, **sem corpo**.

A economia é o corpo inteiro. Só os cabeçalhos trafegam.

Preparando o ambiente

A ferramenta principal é o `curl`, um cliente HTTP de linha de comando.

- **Linux e macOS:** já instalado.
- **Windows 10 e 11:** já instalado, como `curl.exe`.

Atenção, usuários de Windows

No PowerShell, `curl` é apelido de `Invoke-WebRequest`, que tem sintaxe completamente diferente. Escreva `curl.exe` com a extensão.

No `cmd` e no Git Bash, `curl` funciona normalmente. Recomenda-se o **Git Bash** ou um ambiente **WSL**.

Ver a conversa completa

```
curl -v https://example.com
```

A opção `-v` (*verbose*) exibe a mensagem trocada:

- > linhas **enviadas**;
- < linhas **recebidas**;
- * comentários do próprio `curl` sobre a conexão.

Identifique na saída: resolução do nome, handshake TLS, linha de requisição, cabeçalhos enviados, linha de status, cabeçalhos recebidos, corpo.

Somente os cabeçalhos

```
curl -s -o /dev/null -D - https://developer.mozilla.org
```

- -s silencia a barra de progresso;
- -o /dev/null descarta o corpo;
- -D - escreve os cabeçalhos na saída padrão.

```
curl -I https://developer.mozilla.org
```

A opção -I faz algo parecido, porém enviando o método HEAD em vez de GET. São requisições **distintas**, e alguns servidores respondem diferente a cada uma.

Enviar dados: formulário

```
curl -v -d "nome=Ana&curso=TADS" https://postman-echo.com/post
```

A opção `-d`, sozinha, define o método como POST e o cabeçalho `Content-Type: application/x-www-form-urlencoded`.

Este é exatamente o formato que o PHP lerá em `$_POST`, na segunda metade da disciplina.

Cookies com -c e -b

```
curl -s -o /dev/null -c jar.txt \  
    "https://postman-echo.com/cookies/set?turma=ds122"  
cat jar.txt  
curl -s -b jar.txt https://postman-echo.com/cookies  
curl -s          https://postman-echo.com/cookies
```

- -c jar.txt grava os cookies recebidos, fazendo o papel do armazenamento do navegador;
- -b jar.txt reenvia esses cookies.

Com e sem o cookie

```
com -b:    {"cookies":{ ... , "turma":"ds122"}}
sem -b:    {"cookies":{}}
```

O último comando é idêntico ao anterior, exceto pela ausência de -b.

O servidor não guardou nada.

O primeiro comando responde 302, porque o serviço redireciona depois de gravar. E o arquivo jar.txt conterá também cookies definidos pela infraestrutura do site.

Ser o servidor

Até aqui você foi o cliente. Agora inverta os papéis com o `nc` (*netcat*), que abre conexões TCP brutas.

```
nc -l 8080           # macOS, OpenBSD
nc -l -p 8080        # GNU netcat
ncat -l 8080         # Fedora e derivados (nmap-ncat)
```

Com o comando rodando e o terminal aparentemente travado, abra no navegador:

`http://localhost:8080/pagina?a=1&b=2#secao`

O que aparece na requisição do navegador

1. O navegador envia **muito mais cabeçalhos** que o curl: User-Agent, Accept, Accept-Encoding, Accept-Language, Sec-Fetch-* e outros. Compare com a saída do curl -v.
2. A consulta ?a=1&b=2 aparece na linha de requisição. O fragmento #secao **não aparece em lugar nenhum**.
3. O navegador continua carregando, porque **ninguém respondeu**.

Resposta à mão

Digite no terminal do nc e pressione Enter duas vezes após a linha Content-Length:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 38
```

```
<h1>Resposta digitada byte a byte</h1>
```

Confira: <h1> tem 4 bytes, o texto tem 29, </h1> tem 5. Total 38. Se errar para mais, o navegador ficará aguardando bytes que nunca chegam.

Você acabou de operar como servidor web.

De volta às ferramentas do navegador

Abrimos a aba **Rede** no começo da aula sem saber o nome de nada. Volte nela agora: cada coluna e cada campo tem nome, e você já os conhece.

Recomendados: **Firefox** e navegadores baseados em **Chromium**.

Copiar como cURL

Na aba Rede, clique com o botão direito sobre uma requisição e escolha **Copiar como cURL**. Cole no terminal e execute.

Mesma resposta. Navegador e `curl` falam exatamente o mesmo protocolo. A diferença está na quantidade de cabeçalhos.

Para fazer durante a semana

Sem entrega e sem nota

Estes exercícios são de **fixação**. Nada é recolhido nesta aula.

Ainda assim, faça-os. O conteúdo desta aula cai na Prova 1, e digitar os comandos é a única forma de fixar o que vimos aqui.

Sugestão de método: para cada item, anote o **comando** usado, o **trecho relevante** da saída e a **resposta** da pergunta. Guarde num arquivo seu. Vai servir de resumo na semana da prova.

Dúvidas na aula que vem, ou no fórum da UFPR Virtual.

Exercícios 1 a 3

1. Escolha três sites. Para cada um, registre o código de status, a versão do HTTP e o valor de Server, quando presente. Explique as diferenças observadas.
2. Encontre uma URL que responda 301 e outra que responda 404. Para a primeira, mostre a cadeia completa de redirecionamento e informe quantos saltos foram necessários.
3. Envie nome=SeuNome&periodo=2 por GET e por POST para um serviço de eco. Mostre onde os dados aparecem em cada caso. Explique por que a URL do GET fica no histórico e a do POST não, relacionando com segurança e idempotência.

Exercícios 4 a 7

4. Demonstre experimentalmente que o HTTP não guarda estado, usando `-c` e `-b`. Descreva, em no máximo três frases, o que o servidor recebeu em cada requisição.
5. Obtenha o ETag de um recurso, reenvie-o em `If-None-Match` e registre o código e o tamanho do corpo da segunda resposta. Explique a economia produzida.
6. Execute o `nc` como servidor, acesse `http://localhost:8080/teste?x=1#topo` pelo navegador e cole a requisição recebida. Por que `#topo` não aparece, e quem processa esse valor?
7. Meça as fases da conexão com um site brasileiro e outro hospedado fora do país. Compare DNS, TCP e TLS e proponha uma explicação para a diferença.

Os comandos dos itens 5 e 7 não foram vistos em aula. Estão na seção **Mais exemplos com curl**, no material escrito.

Desafio

Escreva um **servidor HTTP mínimo**, em qualquer linguagem, que:

1. abra uma porta TCP;
2. aceite uma conexão;
3. leia a requisição recebida;
4. devolva uma resposta HTTP válida, com Content-Type e Content-Length corretos.

Sem biblioteca pronta

Não use biblioteca de servidor web pronta. Trabalhe diretamente com sockets.

Em Python, cerca de vinte linhas resolvem.

Resumo

- HTTP é um acordo de formato entre cliente e servidor. O cliente sempre inicia.
- Toda mensagem tem linha inicial, cabeçalhos, linha em branco e corpo opcional.
- Métodos se distinguem por serem seguros e idempotentes. Daí decorre o comportamento do navegador, do cache e do histórico.
- Códigos de status separam sucesso, redirecionamento, culpa do cliente e culpa do servidor.
- O protocolo não guarda estado. Cookies, sessões e tokens existem para o cliente carregar o contexto a cada requisição.
- HTTPS acrescenta confidencialidade, integridade e autenticação do servidor, e nada mais.
- Origem é esquema, host e porta juntos. A restrição de mesma origem é do navegador, e o servidor de destino autoriza a exceção.

Bibliografia

- TANENBAUM, A. S. **Redes de Computadores**, cap. 7.3.
- MDN Web Docs. **HTTP**.
<https://developer.mozilla.org/pt-BR/docs/Web/HTTP>
- IETF. **RFC 9110: HTTP Semantics**, 2022.
<https://www.rfc-editor.org/rfc/rfc9110.html>